CASO DEVSHOP - FLUJO DE VALOR Y MEJORA CONTINUA CON ITIL 4

Oscar Juárez Ortiz

Pedro Emiliano Romero Osorio

Emmanuel Baltazar López

Luis Gael Bonilla Paleta

Universidad Madero Puebla

Administración De Procesos Tecnológicos

23/10/2026

Caso DevShop: flujo de valor y mejora continua con ITIL 4 2
Introducción 2
Resumen del caso 2
Parte 1. Análisis del flujo de valor 3
1. ¿Cuál es el flujo de valor? 3
2. ¿Cuáles son sus actividades? 3
3. ¿Dónde existen esperas? 3
4. ¿Dónde podría existir un cuello de botella? 3
5. ¿Qué desperdicios observan? 3
6. ¿Qué información adicional necesitarían? 4
7. ¿Qué mejorarían primero? 4
8. ¿Qué indicador utilizarían? 4
Parte 2. Modelo de mejora continua de ITIL 4 5
1. ¿Cuál es la visión? 5
2. ¿Dónde estamos ahora? 5
3. ¿Dónde queremos estar? 5
4. ¿Cómo llegaremos allí? 5
5. ¡Actuar! ¿Qué debemos hacer? 5
6. ¿Logramos llegar allí? 6
7. ¿Cómo mantenemos el impulso? 6
Conclusión del caso 6

Caso DevShop: flujo de valor y mejora continua con ITIL 4

Introducción

En el entorno actual de desarrollo de software, la velocidad de entrega y la agilidad para responder a los requerimientos del negocio son factores críticos para el éxito operativo. El presente reporte aborda la problemática de DevShop, una organización que enfrenta serias ineficiencias en su flujo de entrega de software. Actualmente, el proceso presenta un tiempo total de entrega (Lead Time) de 16 días desde que el usuario realiza la solicitud hasta que la funcionalidad se encuentra operativa en el entorno de producción.

Un análisis preliminar de este ciclo revela una importante desproporción: la etapa de desarrollo toma únicamente 3 días (equivalente al 19% del tiempo total), mientras que las etapas posteriores de Pruebas (QA) y Aprobaciones / Despliegue consumen 9 días (56% del tiempo global), sin contar las esperas acumuladas en el backlog inicial.

Para resolver este desafío, se aplica la metodología de ITIL 4, enfocándose en el mapeo del Flujo de Valor (Value Stream) y la implementación de las siete etapas del Modelo de Mejora Continua. El objetivo principal es optimizar el flujo global de trabajo, eliminar desperdicios operativos y reducir significativamente el Lead Time, garantizando en todo momento que la calidad, la estabilidad y la seguridad del sistema no se vean comprometidas.

El problema de DevShop no es la velocidad del equipo de desarrollo: Desarrollo ocupa 3 de los 16 días (19 %) y el 81 % restante ocurre fuera de ese equipo. Pruebas y Aprobación concentran 9 días (56 %).

Resumen del caso

Etapa	Duración	% del total
Solicitud	1 día	6 %
Análisis	2 días	13 %
Desarrollo	3 días	19 %
Pruebas	5 días	31 %
Aprobación	4 días	25 %
Despliegue	1 día	6 %

Etapa	Duración	% del total
Total (lead time)	**16 días**	**100 %**

Parte 1. Análisis del flujo de valor

1. ¿Cuál es el flujo de valor?

Es la secuencia de actividades que transforma una demanda (la solicitud de una funcionalidad) en valor (esa funcionalidad funcionando en producción y usada por el cliente). En ITIL 4 es la parte operativa del Sistema de Valor del Servicio (SVS). En DevShop va de la solicitud al despliegue en 16 días:

Solicitud → Análisis → Desarrollo → Pruebas → Aprobación → Despliegue

2. ¿Cuáles son sus actividades?

1. **Solicitud (1 día):** registro y captura de la necesidad (compromiso y planificación).
2. **Análisis (2 días):** comprensión de requisitos, alcance y criterios de aceptación (diseño y transición).
3. **Desarrollo (3 días):** construcción de la funcionalidad (obtención/construcción).
4. **Pruebas (5 días):** validación funcional, de calidad y de seguridad (gestión de validación y pruebas del servicio).
5. **Aprobación (4 días):** autorización formal del cambio (habilitación de cambios).
6. **Despliegue (1 día):** paso a producción (gestión de despliegues y liberaciones).

3. ¿Dónde existen esperas?

El caso habla de "etapas y esperas" dentro de los 13 días, pero no las separa. Con los datos disponibles, las esperas más probables son:

- **Aprobación (4 días):** casi toda es espera, porque aprobar suele ser una decisión rápida que queda en cola hasta que alguien la revisa.
- **Pruebas (5 días):** probable espera por disponibilidad de testers o de ambientes.
- **Entre etapas:** cada transferencia (análisis → desarrollo, desarrollo → pruebas, pruebas → aprobación) suele generar tiempo muerto en cola.

4. ¿Dónde podría existir un cuello de botella?

- **Pruebas (5 días)** es la etapa más larga y el candidato principal. Puede deberse a pruebas manuales, pocos testers o ambientes compartidos.
- **Aprobación (4 días)** es el segundo candidato, por ejemplo un comité de cambios que se reúne una vez por semana o un único aprobador.

Juntas suman 9 de los 16 días (56 %). Mejorar Desarrollo, que ya es una de las etapas más rápidas, no cambiaría casi nada el tiempo total, porque la capacidad del sistema la determina su restricción (Teoría de las Restricciones).

5. ¿Qué desperdicios observan?

Según los desperdicios de Lean que ITIL 4 recoge (*Optimize and automate*, *Keep it simple and practical*):

- **Espera:** tiempo en cola entre etapas, sobre todo en Aprobación y Pruebas.
- **Traspasos (handoffs):** cada cambio de equipo implica pérdida de contexto y retrasos.
- **Sobreprocesamiento:** aprobaciones burocráticas aplicadas igual a cambios de bajo riesgo.
- **Retrabajo:** defectos encontrados al final, en Pruebas, que obligan a volver a Desarrollo.
- **Trabajo en proceso (WIP) excesivo:** muchas funcionalidades abiertas a la vez alargan todos los tiempos.
- **Procesos manuales:** pruebas y despliegue manuales que podrían automatizarse.
- **Falta de visibilidad:** nadie ve el flujo completo, por eso se culpa al equipo de desarrollo.

6. ¿Qué información adicional necesitarían?

- Tiempo de proceso (trabajo real) frente a tiempo de espera en cada etapa, y la eficiencia del flujo (tiempo de valor ÷ lead time).
- Cantidad de solicitudes en cola en cada etapa (WIP) y su variabilidad: ¿siempre 16 días o a veces 30?
- Tasa de retrabajo: cuántas funcionalidades regresan de Pruebas o de Aprobación.
- Quién aprueba, con qué criterios, con qué frecuencia y si todos los cambios exigen el mismo nivel de aprobación.
- Nivel de automatización en pruebas y despliegue, y disponibilidad de ambientes.
- Tipo y riesgo de los cambios: cuántos son estándar o de bajo riesgo.
- Datos de calidad e incidentes posteriores al despliegue, para asegurar que acelerar no degrade el servicio.
- Expectativas del cliente: qué plazo considera aceptable.

7. ¿Qué mejorarían primero?

Se actúa sobre la restricción principal, en este orden:

1. **Aprobación:** es el cambio más rápido y barato. Aplicar *Change Enablement* de ITIL 4 con cambios estándar preautorizados para lo de bajo riesgo, delegar la autoridad de aprobación y reemplazar comités periódicos por aprobaciones continuas.
2. **Pruebas:** automatizar las pruebas de regresión, ejecutarlas desde el desarrollo (*shift-left*) y habilitar ambientes bajo demanda.
3. **Después:** reducir traspasos y WIP, y por último optimizar Análisis y Despliegue (CI/CD).

Se sigue el principio *Start where you are*: se mejora primero lo existente y donde hay más desperdicio, sin rediseñar todo desde cero.

8. ¿Qué indicador utilizarían?

- **Principal: lead time** de solicitud a despliegue, con meta de pasar de 16 días a unos 8-10 días en una primera iteración.
- **Complementarios:**
    - Eficiencia del flujo (tiempo de trabajo ÷ lead time).
    - Tiempo de espera en Pruebas y Aprobación.

- Tasa de fallos de cambios y de retrabajo, como control de calidad.
- Incidentes posteriores al despliegue, como control de estabilidad.

Parte 2. Modelo de mejora continua de ITIL 4

1. ¿Cuál es la visión?

Ser un equipo que entrega valor al cliente de forma rápida, predecible y segura: reducir el tiempo entre la solicitud de una funcionalidad y su despliegue sin comprometer la calidad, la seguridad ni la estabilidad del sistema. Se alinea con el principio *Focus on value*.

2. ¿Dónde estamos ahora?

- Lead time de 16 días: Desarrollo 3 días y el resto 13 días.
- Cuellos de botella probables en Pruebas (5 días) y Aprobación (4 días).
- La dirección percibe que el problema es la velocidad del equipo de desarrollo y no hay visibilidad del flujo completo.
- Falta información sobre esperas, retrabajo y calidad actual; este es el punto de partida que hay que medir bien (línea base).

3. ¿Dónde queremos estar?

Objetivos medibles (SMART), por ejemplo en 3 meses:

- Reducir el lead time de 16 a 10 días o menos, y evaluar llegar a unos 8 después.
- Reducir a la mitad el tiempo de espera en Pruebas y Aprobación.
- Mantener o reducir la tasa de fallos de cambios y de incidentes posteriores al despliegue.
- Tener un tablero con el flujo de valor visible para la dirección y los equipos.

4. ¿Cómo llegaremos allí?

Con un plan por iteraciones, aplicando los principios rectores de ITIL 4:

- **Progress iteratively with feedback:** mejoras pequeñas y medibles, no un cambio masivo.
- **Collaborate and promote visibility:** involucrar a desarrollo, QA, operaciones, seguridad y dirección, y mapear el flujo de valor en conjunto.
- **Think and work holistically:** optimizar todo el flujo y no solo una etapa.
- **Optimize and automate:** automatizar pruebas y despliegue una vez simplificado el proceso.
- **Keep it simple and practical:** eliminar aprobaciones y pasos que no agregan valor.

Prácticas de ITIL 4 involucradas: Change Enablement, Service Validation and Testing, Deployment Management, Release Management, Software Development and Management y Continual Improvement.

5. ¡Actuar! ¿Qué debemos hacer?

Acción	Responsable	Plazo
Mapear el flujo de valor y medir tiempos de proceso y de espera por etapa (línea base)	Líder de mejora + equipos	Semana 1-2
Clasificar cambios por riesgo y definir cambios estándar preautorizados	Change enablement / seguridad	Semana 2-4
Delegar la autoridad de aprobación; sustituir comités periódicos por aprobación continua	Dirección + gestión de cambios	Semana 3-5
Automatizar pruebas de regresión y habilitar ambientes bajo demanda	QA + desarrollo + operaciones	Semana 4-10
Limitar el WIP e implementar un tablero visual (Kanban)	Todos los equipos	Semana 2-4
Automatizar el despliegue (CI/CD) con controles de calidad y seguridad	DevOps	Semana 8-12

6. ¿Logramos llegar allí?

Se evalúa comparando con la línea base:

- ¿El lead time bajó a la meta (10 días o menos)?
- ¿Bajaron las esperas en Pruebas y Aprobación?
- ¿Se mantuvieron o mejoraron la calidad, la seguridad y la estabilidad (fallos de cambios, incidentes, retrabajo)?
- ¿Mejoró la satisfacción del cliente y la percepción de la dirección?

Si mejoró el tiempo pero empeoró la calidad, no se logró el objetivo y se ajusta el plan. Si se logró, se documenta y se identifica la nueva restricción, que probablemente sea otra etapa.

7. ¿Cómo mantenemos el impulso?

- **Revisiones periódicas** (por ejemplo, cada 2 semanas) del tablero de flujo y de los indicadores.
- **Publicar y celebrar los logros** para que la dirección y los equipos vean el valor de la mejora.
- **Registro de mejoras** (*continual improvement register*) con ideas priorizadas por impacto y esfuerzo.
- **Repetir el ciclo:** una vez resuelto un cuello de botella aparece el siguiente, y el modelo vuelve a empezar.
- **Cultura de mejora:** la mejora continua es responsabilidad de todos y se reconoce la participación.
- **Comunicación constante** con la dirección, con datos y no con percepciones.

Conclusión del caso

El problema de DevShop no es la velocidad del equipo de desarrollo. Desarrollo representa 3 de los 16 días (19 %), mientras que Pruebas y Aprobación concentran 9 días (56 %), donde están las restricciones y los desperdicios: esperas, traspasos, retrabajo y burocracia. Presionar a desarrollo no reduciría de forma significativa el lead time y podría afectar la calidad.

Con el enfoque de ITIL 4 se mejora el flujo completo y no una etapa aislada:

1. Se visualiza el flujo de valor y se mide con datos.
2. Se identifica y se actúa sobre la restricción: primero Aprobación mediante cambios estándar, luego Pruebas mediante automatización.
3. Se miden los resultados con el lead time como indicador principal y con indicadores de calidad y estabilidad como control.
4. Se aplica el modelo de mejora continua de forma iterativa para sostener la mejora.

Así DevShop puede entregar valor más rápido sin comprometer la calidad, la seguridad ni la estabilidad, y la dirección pasa de culpar a un equipo a gestionar el sistema completo.